```
             +--------------------------+
             |          CS 2043         |
             | PROJECT 2: USER PROGRAMS |
             |      DESIGN DOCUMENT     |
             +--------------------------+
```

---- GROUP ----

>> Fill in the names and email addresses of your group members.

Fernando H. T. S. D. (240159J) <sithum.24@cse.mrt.ac.lk>

---- PRELIMINARIES ----

>> If you have any preliminary comments on your submission, notes for the
>> TAs, or extra credit, please give them here.

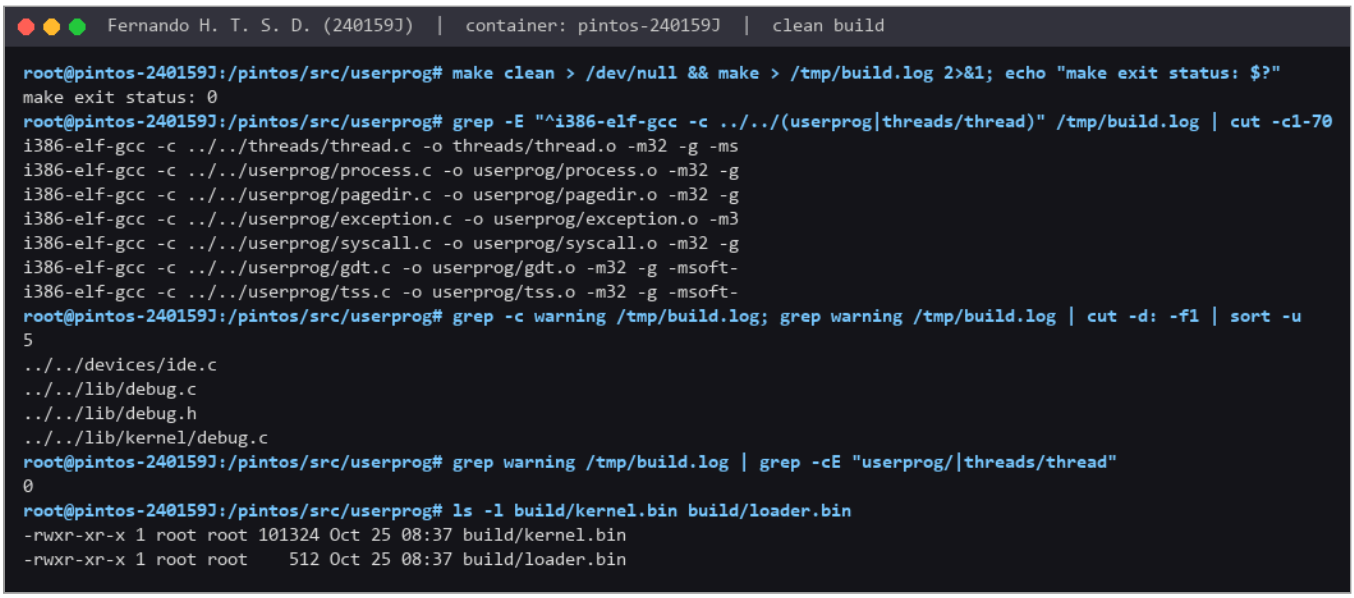
Individual submission for CS 2043 Operating Systems, Semester 3,
Department of CSE, University of Moratuwa.
- Name: Fernando H. T. S. D.
- Index: 240159J
- Lab 2 (User Programs & System Calls)
- All 80 tests passing (`make check` in pintos/src/userprog).

Build and test environment: the course Docker image `pintos`
(i386-elf-gcc 6.2.0, QEMU), run as container `pintos-240159J` with
pintos/src mounted at /pintos/src.
`make clean && make` in pintos/src/userprog builds with no warnings in
any file I changed; the 5 remaining warnings are in unmodified base
files (pintos/src/devices/ide.c, pintos/src/lib/debug.[ch],
pintos/src/lib/kernel/debug.c).

Evidence of testing (in pintos/src/userprog/screenshots/):

01-clean-build.png	Clean build, exit status 0, no warnings in modified files.
02-make-check-summary.png	`make check` summary: "All 80 tests passed."
03-argument-passing-run.png	Manual run of 'args-multiple some arguments for you!' showing argc/argv (repeated spaces collapsed) and the exit message.
04-stack-hex-dump.png	hex_dump of the initial user stack for 'ls-prog -l foo bar' (see A2), taken from a temporary debug build in a scratch copy of the tree; the submitted source has no debug code.
make-check-full.log	Full `make check` console log.

Clean build (01-clean-build.png)



make check summary: All 80 tests passed (02-make-check-summary.png)

Fernando H. T. S. D. (240159J) | container: pintos-240159J | make

```
root@pintos-240159J:/pintos/src/userprog# make check
... (80 tests run under QEMU; per-test output in build/tests/) ...
pass tests/userprog/args-none
pass tests/userprog/args-single
pass tests/userprog/args-multiple
pass tests/userprog/args-many
pass tests/userprog/args-dbl-space
pass tests/userprog/sc-bad-sp
pass tests/userprog/sc-bad-arg
pass tests/userprog/sc-boundary
pass tests/userprog/sc-boundary-2
pass tests/userprog/sc-boundary-3
pass tests/userprog/halt
pass tests/userprog/exit
pass tests/userprog/create-normal
pass tests/userprog/create-empty
pass tests/userprog/create-null
pass tests/userprog/create-bad-ptr
pass tests/userprog/create-long
pass tests/userprog/create-exists
pass tests/userprog/create-bound
pass tests/userprog/open-normal
pass tests/userprog/open-missing
pass tests/userprog/open-boundary
pass tests/userprog/open-empty
pass tests/userprog/open-null
pass tests/userprog/open-bad-ptr
pass tests/userprog/open-twice
pass tests/userprog/close-normal
pass tests/userprog/close-twice
pass tests/userprog/close-stdin
pass tests/userprog/close-stdout
pass tests/userprog/close-bad-fd
pass tests/userprog/read-normal
pass tests/userprog/read-bad-ptr
pass tests/userprog/read-boundary
pass tests/userprog/read-zero
pass tests/userprog/read-stdout
pass tests/userprog/read-bad-fd
pass tests/userprog/write-normal
pass tests/userprog/write-bad-ptr
pass tests/userprog/write-boundary
pass tests/userprog/write-zero
pass tests/userprog/write-stdin
pass tests/userprog/write-bad-fd
pass tests/userprog/exec-once
pass tests/userprog/exec-arg
pass tests/userprog/exec-bound
pass tests/userprog/exec-bound-2
pass tests/userprog/exec-bound-3
pass tests/userprog/exec-multiple
```

```
pass tests/userprog/exec-missing
pass tests/userprog/exec-bad-ptr
pass tests/userprog/wait-simple
pass tests/userprog/wait-twice
pass tests/userprog/wait-killed
pass tests/userprog/wait-bad-pid
pass tests/userprog/multi-recurse
pass tests/userprog/multi-child-fd
pass tests/userprog/rox-simple
pass tests/userprog/rox-child
pass tests/userprog/rox-multichild
pass tests/userprog/bad-read
pass tests/userprog/bad-write
pass tests/userprog/bad-read2
pass tests/userprog/bad-write2
pass tests/userprog/bad-jump
pass tests/userprog/bad-jump2
pass tests/userprog/no-vm/multi-oom
pass tests/filesys/base/lg-create
pass tests/filesys/base/lg-full
pass tests/filesys/base/lg-random
pass tests/filesys/base/lg-seq-block
pass tests/filesys/base/lg-seq-random
pass tests/filesys/base/sm-create
pass tests/filesys/base/sm-full
pass tests/filesys/base/sm-random
pass tests/filesys/base/sm-seq-block
pass tests/filesys/base/sm-seq-random
pass tests/filesys/base/syn-read
pass tests/filesys/base/syn-remove
pass tests/filesys/base/syn-write
All 80 tests passed.
make[1]: Leaving directory '/pintos/src/userprog/build'
```

Files changed from the base code:
  pintos/src/threads/thread.c, pintos/src/threads/thread.h,
  pintos/src/userprog/exception.c, pintos/src/userprog/process.c,
  pintos/src/userprog/process.h, pintos/src/userprog/syscall.c,
  pintos/src/userprog/syscall.h

>> Please cite any offline or online sources you consulted while
>> preparing your submission, other than the Pintos documentation, course
>> text, lecture notes, and course staff.

(none)

ARGUMENT PASSING

---- DATA STRUCTURES ----

>> A1: Copy here the declaration of each new or changed `struct' or
>> `struct' member, global or static variable, `typedef', or
>> enumeration. Identify the purpose of each in 25 words or less.

No new structs, globals, or typedefs were needed. I added two static
helper functions in pintos/src/userprog/process.c:

```
  static bool extract_prog_name (const char *cmdline, char *prog,
                                 size_t size);
```

```
      Copies the first space-delimited token (the program name) into
      a caller-supplied buffer.

  static bool push_arguments (uint8_t *kpage, const char *cmdline,
                              void **esp);
      Writes argv/argc/return-address onto the stack page before it
      gets mapped into the process address space.

I also changed signatures: setup_stack () now takes the command line
as a parameter, and load () receives the full command line in its
first argument (renamed to `cmdline').

---- ALGORITHMS ----

>> A2: Briefly describe how you implemented argument parsing.  How do
>> you arrange for the elements of argv[] to be in the right order?
>> How do you avoid overflowing the stack page?

process_execute () extracts the program name with extract_prog_name ()
(into a 15-byte local buffer), uses it as the thread name, and passes a
copy of the *full* command line to start_process ()/load ().  An empty
or all-space command line returns TID_ERROR.  load () re-extracts the
program name to open the executable (error messages therefore print
only the program name), then calls setup_stack (esp, cmdline).

push_arguments () works on the kernel mapping (kpage) of the stack page
before install_page (), so it never depends on the active page
directory, and a failure only needs to free kpage.
 1. Tokenize a private copy of the command line with strtok_r ()
    (repeated spaces collapse), storing token pointers in a scratch
    page and summing string bytes.
 2. Reject (return false, so load fails cleanly) if argc == 0, argc
    exceeds the scratch capacity, or strings + 3 alignment bytes +
    4*(argc+1) + 12 bytes (argv, argc, return address) exceed PGSIZE.
 3. Copy the strings downward from PHYS_BASE in order argv[0]..argv[n-1]
    recording each user address; because pointers are pushed in
    reverse (argv[argc-1] first, down to argv[0]) after a NULL
    sentinel, argv[0] ends up at the lowest address, i.e. in order.
 4. Word-align, push NULL, the pointers, then argv (address of
    argv[0]), argc and a fake return address 0.  *esp is set to the
    fake return address.

I do the overflow check before writing anything, so a huge command
line can't corrupt memory -- it just makes exec fail cleanly.

Stack layout my code builds for `/bin/ls -l foo bar` (argc = 4,
PHYS_BASE = 0xc0000000).  Because I copy the strings in order
argv[0]..argv[3], "/bin/ls" sits at the top; the spec's example puts
"bar" there instead.  The spec says the string order does not matter,
since the program only reaches the strings through the argv[] pointers.

Address      Name            Type           Value
-------------------------------------------------------------------
0xbffffff8   argv[0][...]    char[8]        "/bin/ls\0"
```

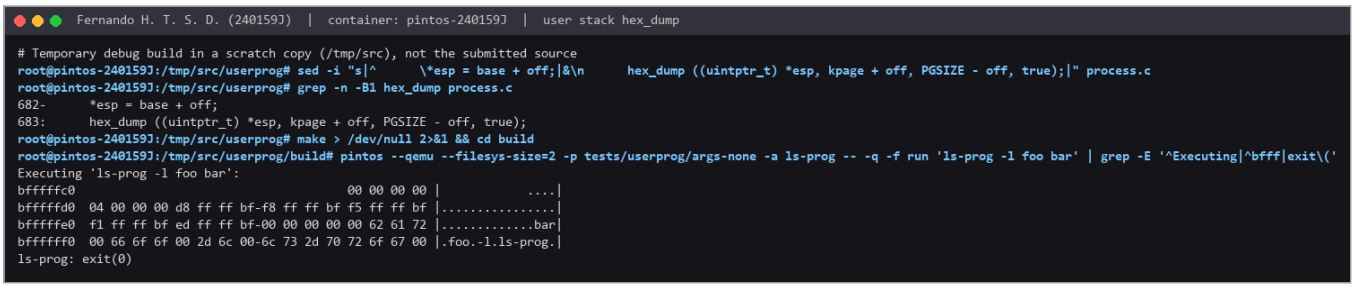
```
0xbffffff5   argv[1][...]    char[3]        "-l\0"
0xbffffff1   argv[2][...]    char[4]        "foo\0"
0xbfffffed   argv[3][...]    char[4]        "bar\0"
0xbfffffec   word-align      uint8_t        0
0xbfffffe8   argv[4]         char *         NULL (0)
0xbfffffe4   argv[3]         char *         0xbfffffed
0xbfffffe0   argv[2]         char *         0xbffffff1
0xbfffffdc   argv[1]         char *         0xbffffff5
0xbfffffd8   argv[0]         char *         0xbffffff8
0xbfffffd4   argv            char **        0xbfffffd8
0xbfffffd0   argc            int            4
0xbfffffcc   return address  void (*) ()    0
```

The initial user stack pointer (*esp) is 0xbfffffcc, the same as in the spec. I checked this table against a real hex_dump of the stack, running the same command with the 7-character program name "ls-prog" because the Pintos file system has no directories ("/bin/ls" cannot be opened). The string lengths are identical, so the addresses are too:

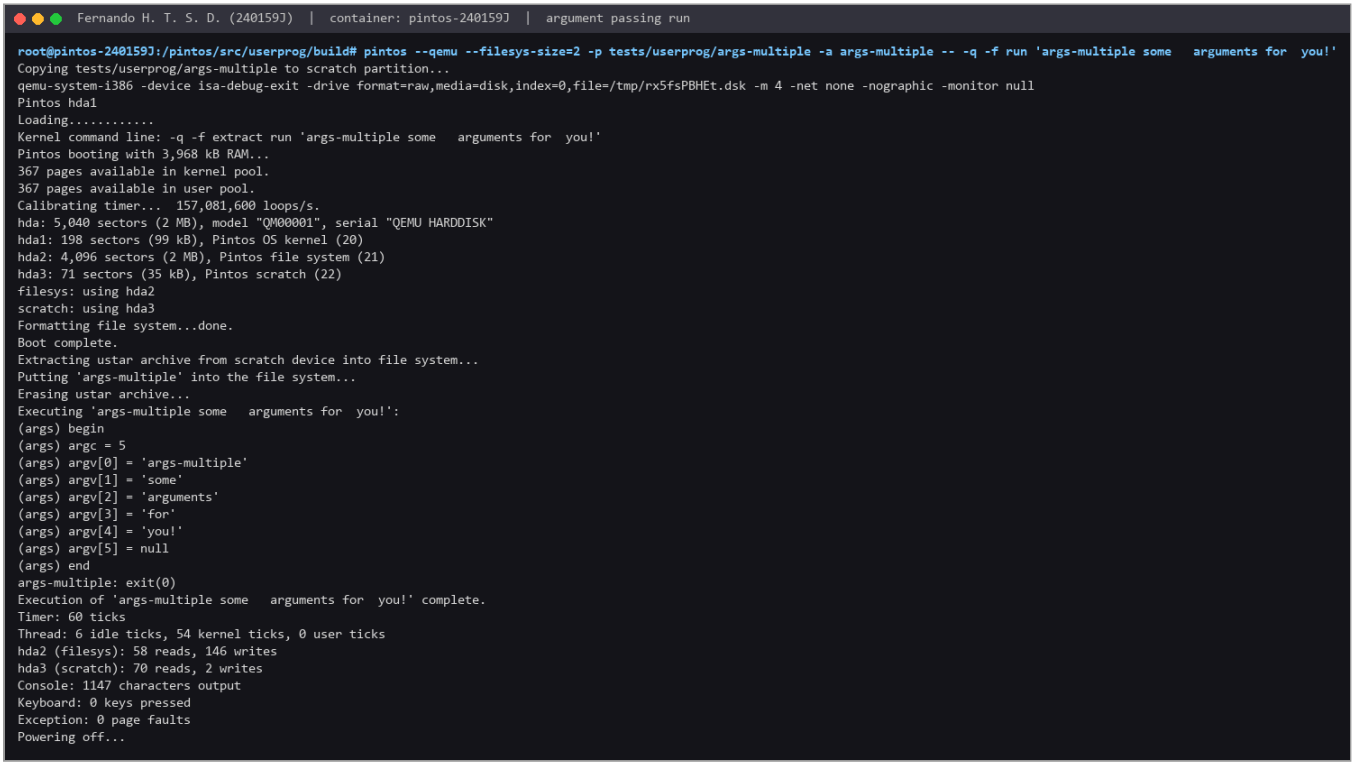
```
bffffc0                                        00 00 00 00 |            ....|
bffffd0  04 00 00 00 d8 ff ff bf-f8 ff ff bf f5 ff ff bf |................|
bffffe0  f1 ff ff bf ed ff ff bf-00 00 00 00 00 62 61 72 |.............bar|
bfffff0  00 66 6f 6f 00 2d 6c 00-6c 73 2d 70 72 6f 67 00 |.foo.-l.ls-prog.|
```

(pintos/src/userprog/screenshots/04-stack-hex-dump.png; a normal run with arguments is in 03-argument-passing-run.png.)

User stack hex_dump for 'ls-prog -l foo bar' (04-stack-hex-dump.png)



Argument passing run (03-argument-passing-run.png)



---- RATIONALE ----

>> A3: Why does Pintos implement strtok_r() but not strtok()?

strtok () keeps its position in a hidden static variable, so it is not

reentrant: two threads (or a thread preempted mid-tokenization) would
corrupt each other's state. Pintos is a preemptive multithreaded
kernel; strtok_r () keeps the state in a caller-supplied save_ptr.

>> A4: In Pintos, the kernel separates commands into an executable name
>> and arguments. In Unix-like systems, the shell does this
>> separation. Identify at least two advantages of the Unix approach.

 1. Less kernel code runs in the privileged domain: a parsing bug in the
    shell only breaks one user process, not the whole kernel.
 2. The shell can do richer processing (quoting, wildcards, variable
    expansion, pipes, redirection) and be replaced or upgraded without
    changing the kernel; the kernel just receives a prepared argv.

                             SYSTEM CALLS
                             ============

---- DATA STRUCTURES ----

>> B1: Copy here the declaration of each new or changed `struct' or
>> `struct' member, global or static variable, `typedef', or
>> enumeration. Identify the purpose of each in 25 words or less.

I introduced the following:

struct child_status (pintos/src/userprog/process.h):
  tid_t tid; int exit_code; bool load_ok; bool load_done;
  char *fn_copy; struct semaphore load_sema; struct semaphore exit_sema;
  int ref_cnt; struct lock ref_lock; struct list_elem elem;
  Shared between parent and child to track exit status, load
  outcome, and synchronization. Reference-counted so either
  side can exit first without leaking memory.

struct thread (pintos/src/threads/thread.h, under USERPROG), new members:
  int exit_code;   Status given to exit(). Defaults to -1.
  struct list fd_list;   Open files for this process (fd_entry nodes).
  int next_fd;   Next fd number to assign (starts at 2).
  struct list children;   List of child_status records.
  struct child_status *my_status;   Points to this thread's own child_status.
  struct file *executable;   Open handle to running binary (write-denied).

struct fd_entry (pintos/src/userprog/syscall.c):
  int fd; struct file *file; struct list_elem elem;
  Represents one open file for a process.

struct lock fs_lock (pintos/src/userprog/syscall.c):
  Global lock that serializes all file system calls.

static const int syscall_nargs[] (pintos/src/userprog/syscall.c):
  Lookup table for how many arguments each syscall takes.

>> B2: Describe how file descriptors are associated with open files.
>> Are file descriptors unique within the entire OS or just within a
>> single process?

File descriptors are unique within a single process, not globally.
Each thread has a linked list of fd_entry nodes. fd_alloc () hands
out the next number (0 and 1 are reserved for the console),
fd_lookup () walks the list, and fd_close () / fd_close_all () clean
up. A new thread starts with an empty list, so children don't
inherit parent descriptors. I chose a linked list over a fixed-size
array to keep struct thread small (only +12 bytes vs +512 for an
array) and to avoid putting an arbitrary cap on how many files a
process can open.

---- ALGORITHMS ----

>> B3: Describe your code for reading and writing user data from the
>> kernel.

I went with the verify-then-dereference approach. I wrote three
helper functions in pintos/src/userprog/syscall.c:

```
  valid_byte (p)      checks p != NULL, p < PHYS_BASE, and the page
                      is actually mapped (pagedir_get_page ()).
  valid_range (p, n)  accepts zero length; otherwise rejects address
                      wrap-around, checks the first and last byte,
                      and one byte on every page in between.
  valid_string (s)    walks to the NUL byte, re-checking at the start
                      and on each page boundary.  Gives up after one
                      page (4096 bytes) to avoid scanning forever.
```

In the handler, I first validate the stack word holding the syscall number and the 1+N argument words (this is why sc-bad-sp and sc-bad-arg are caught before anything gets read). Then I copy the arguments into a kernel-side array and dispatch. Buffers for read/write go through valid_range () and filename strings for exec/create/open/remove go through valid_string (), always before taking fs_lock. If any check fails I call syscall_exit (-1).

As a backstop, I also modified kill () in pintos/src/userprog/exception.c to set exit_code = -1 before calling thread_exit (), so user-mode faults like bad-read or bad-jump still print the "name: exit(-1)" message and clean up properly. I added a check in page_fault () so that a kernel-mode fault on a user address kills the process instead of panicking the kernel.

I considered the get_user ()/put_user () approach with fault recovery but decided against it -- it would be faster, but it needs inline assembly helpers and makes resource cleanup from the fault path harder to get right. The explicit page-table checks felt simpler to reason about and debug.

>> B4: Suppose a system call causes a full page (4,096 bytes) of data
>> to be copied from user space into the kernel. What is the least
>> and the greatest possible number of inspections of the page table
>> (e.g. calls to pagedir_get_page()) that might result? What about
>> for a system call that only copies 2 bytes of data? Is there room
>> for improvement in these numbers, and how much?

Looking at my valid_range (uaddr, size) implementation:
- 4,096-byte copy:
  Least: 2 calls to pagedir_get_page (). This happens when the
  buffer is page-aligned, so valid_byte (start) and valid_byte (last)
  land on the same page and the loop doesn't run.
  Most: 3 calls. When the buffer straddles a page boundary, start
  is on one page and last is on the next, so I get one call for
  start, one for last, and one from the loop for the boundary page.
- Two consecutive bytes:
  Always exactly 2 calls -- valid_byte (start) and valid_byte (last).
  The loop body never executes since there's no intermediate page.
- Room for improvement:
  Yes. A 4,096-byte buffer touches at most two pages, so the
  theoretical minimum is 1 inspection (aligned) and 2 (straddling),
  one per page, i.e. save 1 call in each case; for 2 bytes it is 1
  when both bytes share a page (2 only when they straddle a boundary).
  My valid_range () always checks the first and last byte separately
  even when they are on the same page. Caching the last page checked
  would remove that redundancy. The get_user ()/put_user () approach
  would need no explicit page-table inspection at all, as the MMU
  checks each access and the page-fault handler recovers. I validate
  the entire buffer upfront and kill the process if any part is
  invalid (all-or-nothing).

>> B5: Briefly describe your implementation of the "wait" system call
>> and how it interacts with process termination.

In process_wait (), I scan cur->children looking for a child_status whose tid matches child_tid. If there's no match -- maybe the tid is invalid, or it belongs to a grandchild -- I return -1 right away. When I do find a match, I block on cs->exit_sema until the child exits, read cs->exit_code, and then immediately remove cs from the children list and release my reference. Because I remove it from the list straight away, any second call to wait () on the same tid won't find it and will return -1, so wait () also fails for a tid that is not a direct child or was already waited for.

On the termination side, process_exit () of the child stores the exit code, ups exit_sema (waking a waiting parent) and drops its reference to the child_status; the status therefore outlives the child thread when the parent has not waited yet (see B8).

>> B6: Any access to user program memory at a user-specified address
>> can fail due to a bad pointer value. Such accesses must cause the
>> process to be terminated. System calls are fraught with such
>> accesses, e.g. a "write" system call requires reading the system
>> call number from the user stack, then each of the call's three
>> arguments, then an arbitrary amount of user memory, and any of
>> these can fail at any point. This poses a design and
>> error-handling problem: how do you best avoid obscuring the primary
>> function of code in a morass of error-handling? Furthermore, when
>> an error is detected, how do you ensure that all temporarily
>> allocated resources (locks, buffers, etc.) are freed? In a few
>> paragraphs, describe the strategy or strategies you adopted for
>> managing these issues. Give an example.

To keep error handling out of the main logic, all checking is pulled into the three small predicates valid_byte (), valid_range () and valid_string () (see B3), and every failure goes through a single exit path, syscall_exit (-1), which never returns. The handler first validates the syscall number word and its 1+N argument words and copies the arguments into a kernel-side array, so the individual syscall implementations (syscall_write (), syscall_open (), ...) can assume their arguments are already safe and contain a single "if (!valid_...) syscall_exit (-1);" line each instead of nested error paths.

To avoid leaking resources, I validate every user pointer, string and buffer *before* allocating or acquiring any kernel resource (a file descriptor, malloc memory or fs_lock). When validation fails there is therefore nothing temporary to undo. Whatever the process already owns is released centrally: syscall_exit (-1) (or kill () in pintos/src/userprog/exception.c for hardware faults) leads to process_exit (), which closes every open file in fd_list and the executable, releases the child_status records, ups exit_sema and destroys the page directory. There is no path that can leak.

Example: write (fd, buf, 100) with buf pointing into unmapped memory. The handler validates the stack words, valid_range (buf, 100) fails, and syscall_exit (-1) is called before fs_lock is taken. The process prints "name: exit(-1)" and process_exit () frees its fd_list and child_status records. The same cleanup runs for the bad-jump and bad-read tests, which are killed by the page-fault handler.

---- SYNCHRONIZATION ----

>> B7: The "exec" system call returns -1 if loading the new executable
>> fails, so it cannot return before the new executable has completed
>> loading. How does your code ensure this? How is the load
>> success/failure status passed back to the thread that calls "exec"?

In process_execute (), I allocate a child_status with load_sema = 0 and load_ok = false, then call thread_create (). The parent immediately blocks on sema_down (&cs->load_sema). Over in the child thread, start_process () calls load (). Once load finishes (pass or fail), the child writes the result into cs->load_ok, sets cs->load_done = true, and does sema_up (&cs->load_sema) to wake the parent. The parent then checks cs->load_ok: if true, it returns the child's tid; if false, it removes cs from the children list, drops its reference, and returns TID_ERROR.

There's also a safety net: if the child gets killed before it even finishes loading (e.g., it page-faults during load), process_exit () checks !cs->load_done and ups load_sema anyway, so the parent never deadlocks.

>> B8: Consider parent process P with child process C. How do you
>> ensure proper synchronization and avoid race conditions when P

>> calls wait(C) before C exits? After C exits? How do you ensure
>> that all resources are freed in each case? How about when P
>> terminates without waiting, before C exits? After C exits? Are
>> there any special cases?

Each child_status starts with ref_cnt = 2 (one for the parent, one
for the child), protected by ref_lock. The last side to release it
frees it, so neither side needs to know who exits first. The exit
code is written before exit_sema is upped, and exit_sema is what the
parent blocks on, so there is no race on the exit code.

1. P calls wait(C) before C exits:
   P finds C's entry in cur->children and blocks on exit_sema. When
   C finishes and calls process_exit (), it records the exit code,
   ups exit_sema (waking P), and drops its reference. P wakes up,
   grabs the exit code, removes cs, and calls release_status () to
   free it.

2. P calls wait(C) after C exits:
   C's process_exit () stores its exit code, does sema_up on
   exit_sema, and drops its reference (ref_cnt goes to 1). C's
   thread page and user memory get freed, but the child_status stays
   alive in P's children list. When P eventually calls
   process_wait (), sema_down returns immediately since the semaphore
   was already upped. P reads the exit code, removes cs from the
   list, and releases its reference, freeing the child_status.

3. P terminates without waiting, before C exits:
   When P calls process_exit (), it walks cur->children and calls
   release_status () on each one, dropping ref_cnt to 1. P's thread
   page gets freed. Later when C exits, it sets cs->exit_code, ups
   exit_sema (nobody's listening, but that's fine), and calls
   release_status () which drops ref_cnt to 0 and frees the struct.
   No memory leaks.

4. P terminates without waiting, after C exits:
   C already dropped its reference (ref_cnt == 1), so P's
   release_status () in process_exit () brings it to 0 and frees the
   child_status.

Special cases: (a) wait () on a tid that is not a direct child, or a
second wait () on the same child, returns -1 because the entry is not
in cur->children (see B5). (b) If C is killed during load, the
!load_done check in process_exit () ups load_sema so P's exec () never
deadlocks (see B7). (c) A process killed by the kernel has exit_code
-1, which is what its parent sees. (d) fs_lock is never held while
waiting for a child, and the running executable is closed (re-allowing
writes) before exit_sema is upped, so the parent sees writes enabled
once wait () returns.

---- RATIONALE ----

>> B9: Why did you choose to implement access to user memory from the
>> kernel in the way that you did?

I chose verify-then-dereference: check that the pointer is non-NULL,
below PHYS_BASE and mapped (pagedir_get_page ()) before touching it,
and call syscall_exit (-1) when it is not. It is plain C with no
inline assembly, every check happens at one visible place before any
resource is taken (so cleanup is trivial, see B6), and it is easy to
reason about and debug. The get_user ()/put_user () approach is
faster, because the MMU does the checking, but it needs assembly
helpers and a page-fault handler that can resume the kernel at a
recovery address, which makes releasing locks and buffers from the
fault path harder to get right. The cost of my choice is a few extra
page-table lookups (see B4). kill () and page_fault () in
pintos/src/userprog/exception.c remain as a backstop for any user
fault that slips through.

The file system is not thread-safe, so every file system call is
protected by one global fs_lock. load () holds it for all its file
operations, and fd_close () / fd_close_all () acquire it around

file_close (). User memory is always validated before taking the
lock, and the lock is never held while waiting for a child or creating
a process, so it cannot cause a deadlock.

>> B10: What advantages or disadvantages can you see to your design
>> for file descriptors?

Advantages:
 - A per-thread linked list adds only 12 bytes to struct thread (a
   fixed array would add 512), which matters because the 4 KB thread
   page also holds the kernel stack.
 - There is no arbitrary limit on the number of open files; it is
   only bounded by available memory.
 - fd_close_all () in process_exit () closes everything on every
   termination path, so nothing leaks.
 - Descriptors are per-process, so a process cannot touch another's
   files.
Disadvantages:
 - fd_lookup () is a linear walk, O (n) in the number of open files
   (an array would give O (1)).
 - Each open file costs a malloc'ed fd_entry, and malloc failure in
   open () must be handled (open returns -1).
 - File descriptors are not inherited by children and there is no
   dup ()/dup2 ().

Bad descriptors (negative, not open, writing to stdin, reading from
stdout) return an error value (-1 for read/write/filesize/tell, false
for close, and seek is ignored) instead of killing the process. A bad
fd is a normal runtime error, not a memory violation, so the process
can check the return value as in Unix. A bad *pointer* is a protection
fault and the process has to die. This matches what the tests expect
(read-bad-fd, write-stdin, etc.).

Writes to running executables are denied by keeping the executable
open: after load () succeeds I call file_deny_write () and store the
handle in t->executable instead of closing it. Pintos ties write
denial to the open handle (file_deny_write () increments
inode->deny_write_cnt and file_close () calls file_allow_write ()), so
closing right after loading would allow writes again. Each process
opens its own struct file on the same inode, so for concurrent runs of
one binary (rox-multichild) writes stay denied until every instance
has exited. process_exit () closes cur->executable under fs_lock
before signaling exit_sema (rox-child checks this), and a failed load
() closes the file at its done: label, so no handle leaks.

>> B11: The default tid_t to pid_t mapping is the identity mapping.
>> If you changed it, what advantages are there to your approach?

I did not change it. Each user process is backed by exactly one
kernel thread, so pid_t and tid_t are the same value and no
translation table is needed. process_wait () matches the child by
tid in cur->children, and a separate pid space would add a mapping
structure with no benefit for a single-threaded process model.

                            SURVEY QUESTIONS
                            ================

Answering these questions is optional, but it will help us improve the
course in future quarters. Feel free to tell us anything you
want--these questions are just to spur your thoughts. You may also
choose to respond anonymously in the course evaluations at the end of
the quarter.

>> In your opinion, was this assignment, or any one of the three problems
>> in it, too easy or too hard? Did it take too long or too little time?

(none)

>> Did you find that working on a particular part of the assignment gave
>> you greater insight into some aspect of OS design?

(none)

>> Is there some particular fact or hint we should give students in
>> future quarters to help them solve the problems? Conversely, did you
>> find any of our guidance to be misleading?

(none)

>> Do you have any suggestions for the TAs to more effectively assist
>> students, either for future quarters or the remaining projects?

(none)

>> Any other comments?

(none)